

Resumen Teórico – Práctico 01

Máquinas de Turing

Computabilidad y Complejidad (Teoría de la Computación II)

FCEFQyN – Universidad Nacional de Río Cuarto

Primer Cuatrimestre de 2026

Síntesis de la teoría necesaria para resolver el Práctico 01. Apoyada en el Capítulo 3 del libro de M. Sipser, *Introduction to the Theory of Computation*, y en las láminas de la cátedra ([MaqTuring.pdf](#), [Diag.pdf](#)).

Índice

1. Introducción y motivación histórica	2
2. Definición formal de máquina de Turing	2
2.1. Componentes de una MT	2
2.2. Configuraciones	3
2.3. La relación “cede a”	3
2.4. Configuración inicial, final, cómputo	4
2.5. Lenguaje reconocido por una MT	4
3. Reconocibilidad y decidibilidad	4
3.1. Tres resultados posibles	4
3.2. Relaciones entre las clases	5
3.3. Reconocibilidad por enumeradores	5
4. Niveles de descripción de una MT	6
5. Ejemplos clásicos	6
5.1. M_1 y el lenguaje $\{w\#w\}$ (Sipser 3.9)	6
5.2. M_2 y el lenguaje $\{0^{2^n} \mid n \geq 0\}$ (Sipser 3.7)	6
5.3. M_3 y el lenguaje $\{a^i b^j c^k : i \cdot j = k\}$ (Sipser 3.11)	7
6. Variantes del modelo y robustez	7
6.1. Cinta doblemente infinita	7
6.2. Stay put en lugar de Left	7
6.3. MT con muchas cintas (multicinta)	7
6.4. No determinismo	8
6.5. Restricciones que sí cambian el poder	8
7. La tesis de Church-Turing	9
8. Cardinalidad y existencia de lenguajes no computables	9
8.1. Cardinalidad y conjuntos contables	9

8.2. $\mathbb{R}$ no es contable: el método de la diagonal	9
8.3. Existen lenguajes no computables	10
9. El problema de la detención (preview)	10
10. Subconjuntos infinitos decidibles	10
11. Técnicas para resolver el Práctico 01	11
12. Resumen final	12

1. Introducción y motivación histórica

Las máquinas de Turing (MT) son la formalización clásica de la noción intuitiva de *algoritmo*. Aparecieron en el artículo de Alan Turing de 1936, *On Computable Numbers, with an Application to the Entscheidungsproblem*, motivado por el décimo problema de la lista que Hilbert había planteado en 1900.

El décimo problema de Hilbert. El problema preguntaba si *existe un procedimiento finito* para determinar si un polinomio con coeficientes enteros tiene una raíz integral. Por ejemplo, si se da el polinomio $6x^3yz^2 + 3xz^2$, decidir si existen $x, y, z \in \mathbb{Z}$ tales que el polinomio se anula. El problema en sí mismo es matemático, pero exhibe una dificultad meta-lógica: para responder “no existe tal procedimiento” necesitamos primero *definir formalmente* qué es un “procedimiento”. Es decir, hace falta una definición precisa de la noción intuitiva de algoritmo.

La respuesta de Turing. Turing propuso definir un algoritmo como una máquina abstracta sumamente simple — la que hoy llamamos *máquina de Turing* — y demostró que ya con esa formalización hay problemas que ningún procedimiento finito puede resolver. El décimo problema de Hilbert resulta ser uno de ellos (Matijásevich, 1970).

Intuición. Antes de Turing, el concepto de “algoritmo” era informal: la criba de Eratóstenes o el algoritmo de Euclides eran procedimientos descriptos en lenguaje natural. Esta informalidad bastaba para *exhibir* algoritmos, pero era inútil para *demostrar que un problema no admite ninguno*. Para probar la imposibilidad de un algoritmo hace falta una definición matemática de lo que es un algoritmo. La MT cumple ese rol.

Por qué este Práctico. El Práctico 01 introduce todas las piezas necesarias para hablar de algoritmos formalmente: la definición de MT, sus configuraciones y cómputos, sus variantes principales (cinta doblemente infinita, multicinta, no determinista, enumeradores), y los conceptos de *decidibilidad* y *reconocibilidad*. Este resumen recorre cada uno de esos temas con suficiente profundidad para resolver y entender los ejercicios.

2. Definición formal de máquina de Turing

2.1. Componentes de una MT

Intuitivamente, una máquina de Turing consiste en:

- una *cinta* infinita dividida en celdas, cada una conteniendo un símbolo;

- un *cabezal* que apunta a una celda y puede leer y escribir;
- un *control finito* que, en función del estado actual y del símbolo leído, decide qué escribir, hacia dónde mover el cabezal y a qué estado pasar.

A diferencia de un autómata finito, la MT *escribe* en su cinta y *recorre* la cinta en ambos sentidos; a diferencia de un autómata con pila, su memoria es *ilimitada y de acceso libre*.

Definición 2.1 (7-tupla, Sipser 3.3). Una máquina de Turing es una 7-tupla

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

donde Q, Σ, Γ son conjuntos finitos y:

1. Q es el conjunto de *estados*;
2. Σ es el *alfabeto de entrada*, con $\sqcup \notin \Sigma$;
3. Γ es el *alfabeto de cinta*, con $\Sigma \cup \{\sqcup\} \subseteq \Gamma$;
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ es la *función de transición*;
5. $q_0 \in Q$ es el *estado inicial*;
6. $q_{\text{accept}} \in Q$ es el *estado de aceptación*;
7. $q_{\text{reject}} \in Q$ es el *estado de rechazo*, con $q_{\text{accept}} \neq q_{\text{reject}}$.

El símbolo $\sqcup$ se llama *blanco* y representa una celda vacía de la cinta. Los símbolos en $\Gamma \setminus \Sigma$ son auxiliares y suelen usarse para marcar celdas durante la computación (cruces, marcas, delimitadores).

Intuición. Pensar a $\delta(q, a) = (p, b, D)$ como la regla “si estoy en q y leo a , entonces escribo b , me muevo en dirección D y paso al estado p ”. La dirección $D \in \{L, R\}$ es izquierda o derecha.

2.2. Configuraciones

Para describir el estado dinámico de la máquina (qué tiene en la cinta, en qué estado está, donde apunta el cabezal) se usa el concepto de *configuración*.

Definición 2.2 (Configuración). Una *configuración* de M es una cadena de la forma

$$u q v, \quad u, v \in \Gamma^*, \quad q \in Q,$$

con la siguiente interpretación:

- el contenido de la cinta es uv seguido de blancos;
- el cabezal está sobre el primer símbolo de v ;
- el estado actual es q .

Ejemplo 2.3. La configuración $1011q_701111$ significa: el contenido de la cinta es 101101111 , el cabezal está sobre el primer 0 de 01111 , y el estado actual es q_7 . Esquemáticamente:

$$\boxed{1} \boxed{0} \boxed{1} \boxed{1} \boxed{0} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \sqcup \sqcup \dots$$

$\uparrow q_7$

2.3. La relación “cede a”

Las configuraciones se relacionan por la regla de transición:

Definición 2.4 (Yields, $\rightarrow_M$). Sean C y C' dos configuraciones de M . Decimos que C *cede a* C' en M , notado $C \rightarrow_M C'$, si C' se obtiene de C aplicando una transición de δ . Mas formalmente, sea $a, b, c \in \Gamma$, $u, v \in \Gamma^*$ y $q_i, q_j \in Q$.

- Si $\delta(q_i, b) = (q_j, c, L)$, entonces $uq_i b v \rightarrow_M uq_j a c v$.
- Si $\delta(q_i, b) = (q_j, c, R)$, entonces $uq_i b v \rightarrow_M u c q_j v$.
- En el extremo izquierdo, $q_i b v \rightarrow_M q_j c v$ si la transición es a la izquierda (el cabezal no se mueve, queda en su lugar).
- En el extremo derecho, uq_i es equivalente a $uq_i \sqcup$ y se trata como en los casos anteriores.

2.4. Configuración inicial, final, cómputo

Definición 2.5 (Configuración inicial). La configuración inicial de M con entrada $w = w_1 w_2 \cdots w_n$ es $q_0 w$, es decir: la cinta contiene w en sus primeras n celdas, el cabezal apunta al primer símbolo, el estado es q_0 .

Definición 2.6 (Configuración de aceptación / rechazo). Una configuración $u q v$ es de *aceptación* si $q = q_{\text{accept}}$ y de *rechazo* si $q = q_{\text{reject}}$. Ambas son *configuraciones de detención* (M no ejecuta más transiciones).

Definición 2.7 (M acepta w). M *acepta* la cadena w si existe una sucesión finita de configuraciones $C_0, C_1, \dots, C_k$ tal que:

1. $C_0 = q_0 w$ (configuración inicial);
2. $C_i \rightarrow_M C_{i+1}$ para todo $0 \leq i < k$;
3. C_k es una configuración de aceptación.

La sucesión $C_0, \dots, C_k$ se llama *cómputo* (o *run*) *aceptante* de M sobre w .

2.5. Lenguaje reconocido por una MT

Definición 2.8 (Lenguaje de M). El *lenguaje reconocido* por M es

$$\mathcal{L}(M) = \{ w \in \Sigma^* : M \text{ acepta } w \}.$$

3. Reconocibilidad y decidibilidad

Las MT no siempre se detienen: pueden caer en un bucle. Distinguimos dos clases fundamentales de lenguajes según el comportamiento de la MT que los caracteriza.

3.1. Tres resultados posibles

Cuando M se ejecuta sobre una entrada w , hay exactamente *tres* desenlaces posibles:

- M *acepta* w (alcanza q_{accept}).
- M *rechaza* w (alcanza q_{reject}).
- M *cicla* (no alcanza ningún estado de detención, su cómputo es infinito). También se dice que M *no termina* sobre w .

Definición 3.1 (Turing-reconocible). Un lenguaje $L \subseteq \Sigma^*$ es *Turing-reconocible* (o, equivalentemente, *recursivamente enumerable*, o simplemente *reconocible*) si existe una MT M tal que $\mathcal{L}(M) = L$.

Definición 3.2 (Decidible). Una MT es un *decidor* si *halta* en toda entrada (nunca cicla). Un lenguaje $L \subseteq \Sigma^*$ es *Turing-decidible* (o simplemente *decidible*, o *recursivo*) si existe un decidor M tal que $\mathcal{L}(M) = L$.

Intuición. Reconocer un lenguaje es “poder responder sí con certeza” cuando la respuesta es sí; pero, si la respuesta es no, podemos no obtener nunca esa respuesta (la máquina cicla). Decidir un lenguaje es algo más fuerte: “poder responder sí o no, siempre, en tiempo finito”.

3.2. Relaciones entre las clases

Proposición 3.3 (Inclusión). *Todo lenguaje decidable es reconocible: si M es un decidor para L , en particular $\mathcal{L}(M) = L$.*

La inclusión inversa es *falsa*: existen lenguajes reconocibles pero no decidibles. El ejemplo paradigmático es $A_{TM} = \{\langle M, w \rangle \mid M \text{ es MT y } M \text{ acepta } w\}$, que estudiaremos en el Práctico 02.

Ejemplo 3.4 (Reconocible no decidable: validez de primer orden). Sea $L = \{\varphi : \vdash \varphi\}$, el conjunto de las formulas válidas (teoremas) de la lógica de primer orden. Existe una MT que reconoce L :

1. En la cinta empieza una fórmula φ codificada.
2. La MT genera, una a una, todas las posibles demostraciones formales.
3. Cuando aparece una demostración de φ , acepta.

Si φ es demostrable, eventualmente aparecerá; si no lo es, la máquina nunca termina. Se demuestra (clásico, Gödel/Turing) que L *no* es decidable: no existe un algoritmo que termine siempre con la respuesta correcta sobre la validez.

Observación. La diferencia entre reconocer y decidir es *exactamente* la diferencia entre “sí o no, eventualmente” y “sí o no, en tiempo finito garantizado”. Esa diferencia es responsable de fenómenos como el problema de la detención (Halting Problem).

3.3. Reconocibilidad por enumeradores

Los *enumeradores* dan una caracterización alternativa muy útil de los lenguajes reconocibles. Un enumerador es una MT con dos cintas: una de trabajo y otra (*impresora*) en la que va *imprimiendo* cadenas.

Definición 3.5 (Enumerador). Un *enumerador* es una MT con dos cintas $E = (Q, \Sigma, \Gamma_0, \Gamma_1, q_0, q_{\text{init}}, q_{\text{accept}}, q_{\text{reject}})$ donde Γ_0 es el alfabeto de la cinta de trabajo, Γ_1 el de la cinta de impresión, y q_{print} un estado distinguido tal que, al alcanzarlo, E “imprime” el contenido actual de la cinta de impresión (lo emite como salida) y la limpia para empezar a imprimir la siguiente cadena.

El *lenguaje enumerado* por E es $L(E) = \{s : E \text{ imprime } s \text{ en algún momento}\}$.

Teorema 3.6 (Enumeradores caracterizan reconocibilidad, Sipser 3.21). *Un lenguaje L es Turing-reconocible si y solo si existe un enumerador E con $L(E) = L$.*

Idea de la prueba. ($\Leftarrow$) Si E enumera L , construimos una MT M que reconoce L así: con entrada w , simula E paso a paso; cada vez que E imprime una cadena s , M compara s con w ; si son iguales, M acepta. Si $w \in L$, eventualmente E imprime w y M lo detecta.

($\Rightarrow$) Si M reconoce L , se construye un enumerador E así: enumera todas las cadenas $s_1, s_2, \dots$ de Σ^* y, en la iteración i , simula M por i pasos sobre cada s_j con $j \leq i$. Cuando M acepta alguna s_j , E la imprime. Esto evita ciclar en una s_j que cicla en M : usa una estrategia de *dovetailing*. $\square$

Intuición. Un enumerador puede imprimir las cadenas en cualquier orden y con repeticiones. La caracterización dice que “poder reconocer” (responder sí a las cadenas del lenguaje) es lo mismo que “poder enumerarlas”.

4. Niveles de descripción de una MT

Sipser (Sección 3.3) distingue **tres niveles** para describir máquinas de Turing. La elección del nivel depende de qué tan detallado deba ser el análisis.

1. **Descripción formal:** se especifican explícitamente los siete componentes de la 7-tupla, en particular la función δ como tabla o diagrama de estados. Es el nivel más detallado y riguroso, pero impráctico para máquinas grandes.
2. **Descripción a nivel implementación:** se da en lenguaje natural, indicando cómo la máquina mueve el cabezal y modifica la cinta paso a paso (escribe, marca, recorre, vuelve, etc.), sin listar los estados ni la δ . Es el nivel preferido para los ejercicios.
3. **Descripción de alto nivel:** se da el algoritmo a nivel lógico, ignorando cómo se gestionan la cinta y el cabezal. Es el nivel del “pseudocódigo” y se justifica por la *tesis de Church-Turing*.

Intuición. Es como programar: el nivel formal es ensamblador, el de implementación es C, y el de alto nivel es Python o pseudocódigo. Todos describen el mismo algoritmo, pero a distinto detalle.

5. Ejemplos clásicos

Repasamos algunas MT de referencia que aparecen en Sipser y en las láminas de la cátedra. Estos ejemplos ilustran técnicas que se aplican directamente en los ejercicios.

5.1. M_1 y el lenguaje $\{w\#w\}$ (Sipser 3.9)

Sea $A = \{w\#w \mid w \in \{0,1\}^*\}$. La MT M_1 que decide A funciona en “zigzag”: para cada símbolo a la izquierda del $\#$, marca el correspondiente a su derecha. Si encuentra discrepancia, rechaza; si los consume todos sin discrepancias, acepta.

Técnica clave: **marcar celdas** con símbolos auxiliares ($x, y \in \Gamma \setminus \Sigma$) para recordar qué ya se procesó.

5.2. M_2 y el lenguaje $\{0^{2^n} \mid n \geq 0\}$ (Sipser 3.7)

M_2 decide $A = \{0^{2^n}\}$, las cadenas de ceros cuya longitud es potencia de 2. La estrategia:

1. recorrer la cinta tachando uno de cada dos 0s (con x);
2. si tras la pasada queda un único 0, aceptar;
3. si la cantidad de 0s sin tachar es impar y mayor que 1, rechazar;
4. si es par, volver al inicio y repetir.

Cada iteración divide a la mitad la cantidad de 0s; si la longitud no es potencia de 2, en alguna iteración aparecerá paridad impar.

Técnica clave: **contador de paridad implementado en estados** (q_3, q_4 alternan).

5.3. M_3 y el lenguaje $\{a^i b^j c^k : i \cdot j = k\}$ (Sipser 3.11)

M_3 usa la estrategia: por cada a , “shuttlea” entre los b y los c tachando uno de cada para emparejarlos en cantidad j , y así descontar j veces de los c por cada a . Si al consumir todas las a los c quedaron exactamente vacíos, acepta. Esta es la técnica que extenderemos en el Ejercicio 3 del práctico.

6. Variantes del modelo y robustez

Una característica notable de las MT es su **robustez**: muchas modificaciones “naturales” del modelo no cambian la clase de lenguajes reconocibles. Esto justifica la tesis de Church-Turing y nos permite elegir, en cada situación, la variante más conveniente.

6.1. Cinta doblemente infinita

Definición 6.1 (MT con cinta doblemente infinita). Una MT con cinta doblemente infinita es idéntica al modelo estándar, salvo que su cinta es infinita en *ambos* sentidos (las celdas están indexadas por $\mathbb{Z}$ en lugar de $\mathbb{N}$). Inicialmente la entrada $w_0 \cdots w_{n-1}$ ocupa las celdas $0, 1, \dots, n-1$ y el resto contiene blancos.

Teorema 6.2 (Equivalencia, Problema 3.11 de Sipser). *Las MT con cinta doblemente infinita reconocen exactamente los mismos lenguajes que las MT estándar.*

Idea de la prueba. La inclusión estándar $\subseteq$ doblemente-infinita es trivial (basta no usar la zona de la izquierda). La recíproca usa la técnica de “doblar la cinta”: se codifican las celdas $i \in \mathbb{Z}$ en celdas $e(i) \in \mathbb{N}$ con

$$e(i) = \begin{cases} 2i & i \geq 0, \\ -2i - 1 & i < 0, \end{cases}$$

de modo que las posiciones no negativas van a celdas pares y las negativas a impares. Una marca distinguida indica la posición simulada del cabezal. $\square$

Este teorema corresponde directamente al Ejercicio 4 del práctico.

6.2. Stay put en lugar de Left

Sipser (Problema 3.13) considera una variante donde el cabezal puede moverse a la derecha o quedarse en su lugar (pero no ir a la izquierda). **Esta variante no es equivalente** a la MT estándar: es estrictamente más débil. Es un buen contraste para entender que no toda variación del modelo preserva el poder.

6.3. MT con muchas cintas (multicinta)

Definición 6.3 (Multicinta). Una MT con k cintas es una tupla

$$M = (Q, \Sigma, \{\Gamma_i\}_{0 \leq i < k}, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

donde cada cinta i tiene su propio alfabeto Γ_i y su propio cabezal, y la función de transición es

$$\delta : Q \times \Gamma_0 \times \cdots \times \Gamma_{k-1} \rightarrow Q \times \Gamma_0 \times \cdots \times \Gamma_{k-1} \times \{L, R\}^k.$$

La entrada se ubica en la cinta 1; el resto comienzan en blanco.

Teorema 6.4 (Multicinta $\equiv$ una cinta, Sipser 3.13). *Toda MT con k cintas tiene una MT equivalente con una sola cinta.*

Idea de la prueba. Sea M con k cintas. Construimos S con una sola cinta que codifica las k cintas en bloques separados por un nuevo símbolo $\#$:

$$\# u_1 \# u_2 \# \cdots \# u_k \#.$$

Para indicar las posiciones de los k cabezales, S marca un símbolo de cada bloque con un punto (*dotted*). Para simular un paso de M :

1. S recorre toda la cinta y memoriza en su control finito los k símbolos marcados.
2. Calcula δ_M con esos símbolos.
3. Vuelve a recorrer la cinta y, en cada bloque, escribe el nuevo símbolo y mueve la marca.
4. Si una marca debe avanzar más allá del fin de su bloque, S “expande” el bloque desplazando todo a la derecha.

□

Corolario 6.5. *L es Turing-reconocible si y sólo si lo reconoce alguna MT multicinta.*

6.4. No determinismo

Definición 6.6 (MT no determinista, NTM). Una NTM se obtiene cambiando la signatura de δ :

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\}).$$

En cada configuración, hay un conjunto finito de transiciones posibles. La máquina *acepta* w si *alguna* rama de su árbol de ejecución alcanza q_{accept} .

Teorema 6.7 (Determinismo $\equiv$ no determinismo, Sipser 3.16). *Para toda NTM existe una MT determinista equivalente.*

Idea de la prueba. Se simula la NTM con un **BFS por capas** sobre el árbol de configuraciones: la MT determinista visita las configuraciones de profundidad 1, luego de profundidad 2, etc. Si alguna rama acepta, el BFS la encontrará en tiempo finito. Si ninguna acepta, el BFS continúa indefinidamente (esto refleja que la NTM puede tener ejecuciones que no terminan). □

Observación. El no determinismo no agrega poder de reconocimiento, pero a veces *simplifica* el diseño de una MT (especialmente en la teoría de complejidad).

6.5. Restricciones que sí cambian el poder

No todas las variaciones preservan poder. Por ejemplo, una MT que **no puede escribir sobre el input** (Problema 3.20 de Sipser, y Ejercicio 6 del práctico) reconoce **solo lenguajes regulares**: el modelo se reduce a un autómata finito de dos vías (2DFA), que es equivalente a un DFA por el clásico teorema de Rabin–Scott / Shepherdson.

7. La tesis de Church-Turing

Teorema 7.1 (Tesis de Church-Turing). *La noción intuitiva de algoritmo coincide con la noción formal de computable por una MT.*

No es un teorema en sentido estricto (la noción intuitiva no es formalizable); es una *tesis científica* sostenida por evidencia acumulativa: todas las formalizaciones razonables propuestas (MT, λ -cálculo de Church, funciones recursivas de Gödel-Kleene, máquinas de Post, RAM, etc.) resultan **equivalentes en poder**.

Intuición. La tesis es la que justifica que, en niveles altos de descripción, podamos “hablar de algoritmos” sin referirnos explícitamente a una MT: si algo se puede describir como un procedimiento finito y bien definido, entonces existe una MT que lo computa. Esto es lo que permite que el Ejercicio 3 del práctico se resuelva con descripciones tipo Sipser sin escribir 100 líneas de δ .

8. Cardinalidad y existencia de lenguajes no computables

Esta sección resume el contenido de `Diag.pdf` y motiva la existencia de lenguajes no reconocibles — y por ende, no decidibles — por argumentos de cardinalidad.

8.1. Cardinalidad y conjuntos contables

Definición 8.1 (Igualdad de cardinalidad). Dos conjuntos A y B tienen la misma cardinalidad, $A \cong B$, si existe una función biyectiva $f : A \rightarrow B$.

Definición 8.2 (Conjunto contable). A es *contable* si $A \cong \mathbb{N}$. Equivalentemente, los elementos de A pueden listarse en una sucesión $a_1, a_2, a_3, \dots$.

Ejemplo 8.3. $\mathbb{Z}$ es contable, vía $f(n) = \lceil n/2 \rceil$ si n impar, $f(n) = -n/2$ si n par.

Ejemplo 8.4. $\mathbb{Q}$ es contable: la enumeración diagonal de $(p, q) \in \mathbb{N} \times \mathbb{N}^+$ recorre todos los pares y, eliminando los no reducidos, lista todos los racionales positivos. Combinando con los negativos y el cero se obtiene una enumeración de $\mathbb{Q}$.

8.2. $\mathbb{R}$ no es contable: el método de la diagonal

Teorema 8.5 (Cantor). $\mathbb{R}$ no es contable.

Demostración. Basta mostrar que $[0, 1)$ no es contable. Supongamos por absurdo que $[0, 1) \cong \mathbb{N}$ y enumeremos los elementos de $[0, 1)$ por su expansión decimal:

$$r_1 = 0.d_1^1 d_2^1 d_3^1 \dots \quad r_2 = 0.d_1^2 d_2^2 d_3^2 \dots \quad r_3 = 0.d_1^3 d_2^3 d_3^3 \dots \quad \dots$$

Construimos un número $r = 0.\bar{d}^1 \bar{d}^2 \bar{d}^3 \dots$ con

$$\bar{d}^i = \begin{cases} 1 & d_i^i \neq 1, \\ 0 & d_i^i = 1. \end{cases}$$

Por construcción r difiere de r_i en la i -ésima cifra decimal, para todo i . Por lo tanto $r \notin \{r_1, r_2, \dots\}$, contradiciendo que la enumeración era completa. $\square$

Este es el **método de la diagonal de Cantor**, una herramienta clave para probar resultados de no computabilidad.

8.3. Existen lenguajes no computables

Teorema 8.6. *Existen lenguajes $L \subseteq \{0, 1\}^*$ que no son reconocibles por ninguna MT.*

Demostración. ■ El conjunto de descripciones de MT $\{\langle M \rangle : M \text{ es MT}\}$ es un subconjunto contable de $\{0, 1\}^*$. Como $\{0, 1\}^*$ es contable, su cardinalidad es $\aleph_0$.
■ El conjunto de *lenguajes* sobre $\{0, 1\}^*$ es $\mathcal{P}(\{0, 1\}^*)$. Por el teorema de Cantor, $\mathcal{P}(\{0, 1\}^*)$ tiene cardinalidad $2^{\aleph_0} > \aleph_0$.

Cada MT reconoce un lenguaje, así que la cantidad de lenguajes reconocibles es a lo sumo $\aleph_0$. Como hay $2^{\aleph_0}$ lenguajes en total, debe haber lenguajes no reconocibles (de hecho, “casi todos” lo son). $\square$

Intuición. El argumento es *no constructivo*: dice que existen lenguajes no reconocibles pero no exhibe ninguno. La construcción de un ejemplo explícito (como $\overline{A_{TM}}$, Capítulo 4 de Sipser) requiere diagonalización como en la prueba de Cantor.

9. El problema de la detención (preview)

Aunque pertenece propiamente al Práctico 02, es útil introducir el problema de la detención para entender por qué existe la distinción entre reconocible y decidable.

Definición 9.1. $A_{TM} = \{\langle M, w \rangle \mid M \text{ es una MT que acepta } w\}$.

Teorema 9.2 (Indecibilidad de A_{TM} , Turing 1936). A_{TM} no es decidable.

Idea (diagonalización). Supongamos que existe H que decide A_{TM} . Construimos una nueva MT D que, con entrada $\langle M \rangle$:

1. simula H sobre $\langle M, \langle M \rangle \rangle$;
2. si H acepta, D rechaza; si H rechaza, D acepta.

Ahora corremos D con entrada $\langle D \rangle$:

- Si D acepta $\langle D \rangle$, entonces H rechazó $\langle D, \langle D \rangle \rangle$, es decir D no acepta $\langle D \rangle$. Contradicción.
- Si D rechaza $\langle D \rangle$, entonces H aceptó $\langle D, \langle D \rangle \rangle$, es decir D acepta $\langle D \rangle$. Contradicción.

Por lo tanto, H no existe. $\square$

Observación. A_{TM} sí es reconocible: la máquina universal U que simula M con entrada w y acepta si M acepta es un reconocedor para A_{TM} . Por lo tanto A_{TM} es un lenguaje **reconocible pero no decidable**, ejemplo concreto de la diferencia entre las dos clases.

10. Subconjuntos infinitos decidibles

Este resultado conecta directamente con el Ejercicio 5 del práctico.

Teorema 10.1 (Sipser, Problema 3.19). *Todo lenguaje infinito Turing-reconocible tiene un subconjunto infinito decidable.*

Esquema. Sea A infinito y reconocible. Por el Teorema 3.6 existe un enumerador E para A . Fijamos el orden *shortlex* sobre Σ^* (longitud, luego lexicográfico). Construimos inductivamente:

- b_1 : primera cadena que imprime E .
- b_{i+1} : primera cadena que imprime E con $b_{i+1} > b_i$ en *shortlex*.

Como A es infinito, siempre existe alguna cadena de A mayor que b_i , y como E enumera A , eventualmente la imprime. La sucesión $b_1 < b_2 < \dots$ es estrictamente creciente, infinita, y está contenida en A .

Sea $B = \{b_1, b_2, \dots\}$. Para decidir B , dada x , generamos $b_1, b_2, \dots$ hasta encontrar el primer $b_k \geq x$:

- si $b_k = x$, aceptamos;
- si $b_k > x$, rechazamos.

Bajo *shortlex*, hay finitas cadenas $\leq x$, así que en finitos pasos encontramos $b_k \geq x$. Luego B es decidable. $\square$

11. Técnicas para resolver el Práctico 01

Resumimos las herramientas conceptuales y técnicas que aparecen una y otra vez en los ejercicios.

T1. Convencion para escribir configuraciones

Para el Ejercicio 2 (configuraciones de M_2), basta aplicar paso a paso la regla $C \rightarrow_M C'$ usando la tabla de δ . Una configuración uqv codifica simultáneamente cinta, posición del cabezal y estado. El blanco $\sqcup$ sólo aparece a la derecha del segmento “activo” de la cinta o como sentinela del extremo izquierdo.

T2. Marcado y emparejamiento

Es la técnica central del Ejercicio 3 (deciders para $\{w : \#_0(w) = \#_1(w)\}$ y $\{w : \#_0(w) = 2\#_1(w)\}$). Se usan símbolos auxiliares (clásicamente x para “0 usado” e y para “1 usado”) para emparejar símbolos del input. Cada iteración toma un símbolo no marcado y busca su(s) contraparte(s); termina cuando ya no hay símbolos sin marcar.

Plantilla:

- Volver al inicio.
- Buscar el primer símbolo no marcado.
- Si no hay, aceptar (o pasar a una fase final de verificación).
- Si hay, marcarlo y buscar la contraparte. Si no existe, rechazar.
- Reiniciar.

T3. Simulación de variantes

Para el Ejercicio 4 (cinta doblemente infinita) y para los algoritmos de traducción entre modelos:

- **Doblemente infinita** → **estándar**: usar la codificación $e : \mathbb{Z} \rightarrow \mathbb{N}$, $e(i) = 2i$ si $i \geq 0$, $e(i) = -2i - 1$ si $i < 0$.
- **Estándar** → **doblemente infinita**: reservar un marcador de extremo izquierdo $\vdash$ y nunca cruzarlo.
- **Multicinta** → **una cinta**: codificar las k cintas en bloques separados por $\#$, con marcas para los k cabezales.
- **Una cinta** → **multicinta**: usar sólo la cinta 1 y dejar las restantes en blanco.

T4. Enumeración en orden creciente

Para el Ejercicio 5: dado un enumerador E de A , extraer la sucesión estrictamente creciente $b_1 < b_2 < \dots$ tomando, sucesivamente, la primera cadena impresa que sea mayor que la anterior. Decidir el subconjunto B generado por esa sucesión usando la finitud de $\{y \leq x\}$ bajo shortlex.

T5. Reducción a un modelo más débil

Para el Ejercicio 6: si una MT no puede escribir sobre el input, su comportamiento sobre la zona del input se reduce a movimientos del cabezal sin alterar la cinta. Esto la convierte en un **2DFA**, y por Rabin–Scott / Shepherdson los lenguajes reconocidos por 2DFA son los regulares.

12. Resumen final

Hemos cubierto los conceptos centrales para abordar el Práctico 01:

- La **definición formal** de MT como 7-tupla $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ y la noción de *configuración* y *cómputo*.
- La distinción entre **lenguajes reconocibles** (existe MT que los acepta) y **decidibles** (existe decidor, MT que halta siempre). La inclusión $\text{DEC} \subsetneq \text{REC}$ es estricta.
- Las **descripciones** de MT en tres niveles (formal, implementación, alto nivel), justificadas por la tesis de Church–Turing.
- Las **variantes** principales del modelo: cinta doblemente infinita, multicinta, no determinista, enumeradores. Todas *equivalentes* en poder; pero pequeñas restricciones (como “no escribir sobre el input”) sí pueden reducir el poder a la clase regular.
- La **tesis de Church–Turing**, que identifica algoritmo con cómputo por MT.
- La **existencia de lenguajes no computables** (Cantor) y un **ejemplo concreto** indecidible reconocible: A_{TM} .
- El **teorema sobre subconjuntos infinitos decidibles** y las **técnicas** para los ejercicios.

Hilo conductor. La idea general detrás del Práctico 01 es: *construir intuición operativa con las MT, observar que pequeñas modificaciones del modelo no cambian su poder (robustez) salvo cuando removemos alguna capacidad clave (como escribir), y entender por qué existe la distinción fundamental entre reconocer y decidir, distinción que será el motor del Práctico 02 sobre indecidibilidad y reducciones.*

Fin del resumen teórico. Para profundizar, leer el Capítulo 3 completo de Sipser y revisar los ejercicios del Práctico 01.